붕어빵(fishbun)

[문제] k개의 빵틀(frame)이 일렬로 배치된 기계로 붕어빵을 구워내려고 한다. 따라서 이 붕어빵 틀은 배열(array) **Fish[k]**로 표현된다. 붕어빵은 앞 뒷면이 서로 다르게 구별되도록 만들어져 있다. 이 빵틀은 좀 독특하게 구성되어 있어서 연속된 구간의 모든 빵을 뒤집을 수 있다. 물론 1개의 칸도 뒤집을 수도 있다.

예를 들어 특정 일부 구간 **Fish[i:j]**를 뒤집으면 j번째 항목은 i번에 오고 i번째 항목은 j번으로 이동한다. 즉 수평 방향으로 배열의 값이 세로축으로 대칭(symmetric) 이동하는 것이다. 이러한 작업을 flipping이라고도 한다. 아래 그림을 이용해서 설명해보자. 그림은 12개의 틀로 이루어진 붕어빵 기계를 배열로 보여주고 있다. 첫 줄에서 1부터 12까지 표시된 번호는 각 틀의 index를 의미한다. 처음 각 틀에 들어간 빵은 1부터 12까지의 정수로 표시되어 있다. 만일 번호 k인 붕어빵이 Flip으로 한번 뒤집히면 이 숫자는 $-k$로 표시된다. 따라서 한 번 더 뒤집히면 $-(-k)=k$로, 원래 값으로 되돌아온다. 초기 상태에는 각 빵은 인덱스의 번호로 지정된다.

1	2	3	4	5	6	7	8	9	10	11	12
1	2	3	4	5	6	7	8	9	10	11	12

flip[4,9]: 만일 위 상태에서 구간 [4,9]를 뒤집으면 빵틀은 다음과 같이 변한다.

1	2	3	4	5	6	7	8	9	10	11	12
1	2	3	-9	-8	-7	-6	-5	-4	10	11	12

flip[6,11]: 이어서 구간 [6,11]을 뒤집으면 다음과 같이 변한다.

1	2	3	4	5	6	7	8	9	10	11	12
1	2	3	-9	-8	-11	-10	4	5	6	7	12

flip[2,4]: 다시 구간 [2,4] 를 뒤집으면 다음과 같이 변한다.

1	2	3	4	5	6	7	8	9	10	11	12
1	9	-3	-2	-8	-11	-10	4	5	6	7	12

이 flip을 역순으로 전개하면 우리는 붕어빵 틀을 원래의 상태로 재구성할 수 있다. 예를 들어 빵틀의 상태가 [1, 9, -3, -2, -8, -11, -10, 4, 5, 6, 7, 12]인 상태에서 **flip[2,4]** → **flip[6,11]** → **flip[4,9]** 수행하면 [1, 2, 3, .. 12]를 만들 수 있고 이 상태에서 붕어빵을 모두 꺼낸다.

여러분에게는 어떤 붕어빵틀의 상태가 정수 배열로 주어진다. 여러분은 이 상태에서 최소한의 **flip[]** 작업을 이용하여 최초의 상태 [1, 2, 3, ... k]를 만들어야 한다. 즉 최종 입력 상태를 읽어서 원상 복구에 필요한 최소한의 flip 작업의 수를 계산해야 한다. 만일 1번의 flip으로 충분하다면 "one", 최소 2번으로 가능하다면 "two", 그리고 3번 이상의 flip이 필요하다면 "over"를 정답으로 출력해야 한다. 단 한 번도 뒤집지 않는 경우는 입력으로 들어오지 않으므로 정답은 반드시 **{one, two, over}** 중 하나가 되어야 한다.

[입출력] 입력과 출력은 표준 입출력 파일인 **stdin과 stdout을** 사용한다. 입력 파일 stdin의 첫 줄에는 붕어빵틀의 크기 k가 정수로 주어진다. 각 test case마다 5개씩의 subcases가 준비되어 있다. 즉 이어지는 5개의 줄에는 붕어빵틀의 최종 상태를 나타내는 k개의 정수가 주어진다. 각 정수는 하나 이상의 공백으로 분리되어 있다. 당연히 입력 정수는 $-k$부터 k범위의 서로 다른 정수로 구성되어 있다. 따라서 여러분은 {"one", "two", "over" } 중에서 정답을 골라 순서대로 5개의 줄에 출력해야 한다. 단 5개를 모두 맞혀야 하나의 test case를 맞춘 것으로 처리된다. k의 범위는 $5 \le k \le 100{,}000$ 이다.

[예제]

stdin	stdout
6 //빵틀의 갯수 1 2 -4 -3 5 6 1 2 4 -3 5 6 -3 -2 6 1 -5 -4 4 5 -1 -6 2 3 6 1 -5 -4 2 3	one two over over over
12 //빵틀의 갯수 1 2 3 4 5 6 7 8 9 -10 11 12 1 -2 3 4 5 6 7 8 9 -10 11 12 1 -2 3 -4 5 -6 7 -8 9 10 -11 12 1 9 -3 -2 -8 -11 -10 4 5 6 7 12 1 -12 -11 -10 7 8 9 -6 -5 -4 -3 -2	one two over over two

[제한조건] 프로그램 이름은 **fishbun.{c, cpp, py}**이다. 과제 제출 시작 시간과 마감시간, 최대 제출 횟수, 수행시간(CPU time), 제출 프로그램의 최대 사용 token의 수는 NESPA에서 확인할 수 있으므로 자주 NESPA를 방문하여 확인하기 바란다. 'token'은 공백으로 분리되는 문자열(string)로 생각하면 된다. 또는 NESPA에서 코드를 넣어 제출전에 그 갯수를 확인해볼 수도 있다. 단 코드의 comment 문장은 이 계산에 포함되지 않는다.

만일 GPT등을 활용한 경우에는 이 사실을 코드에 comment로 표시해야 한다. 그렇지 않으면 억울하게 표절로 판정받을 수 있다. 코드에 한글을 사용한 경우 인코딩 방법에 따라서 오류가 날 수 있으므로 가능하면 영어를 사용하는 것을 권장한다.